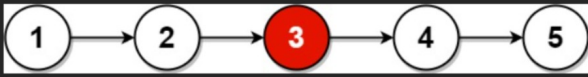


Day 21 :- Lecture Example on Linked List

Lectcode Pbm No 876

Example 1:



head

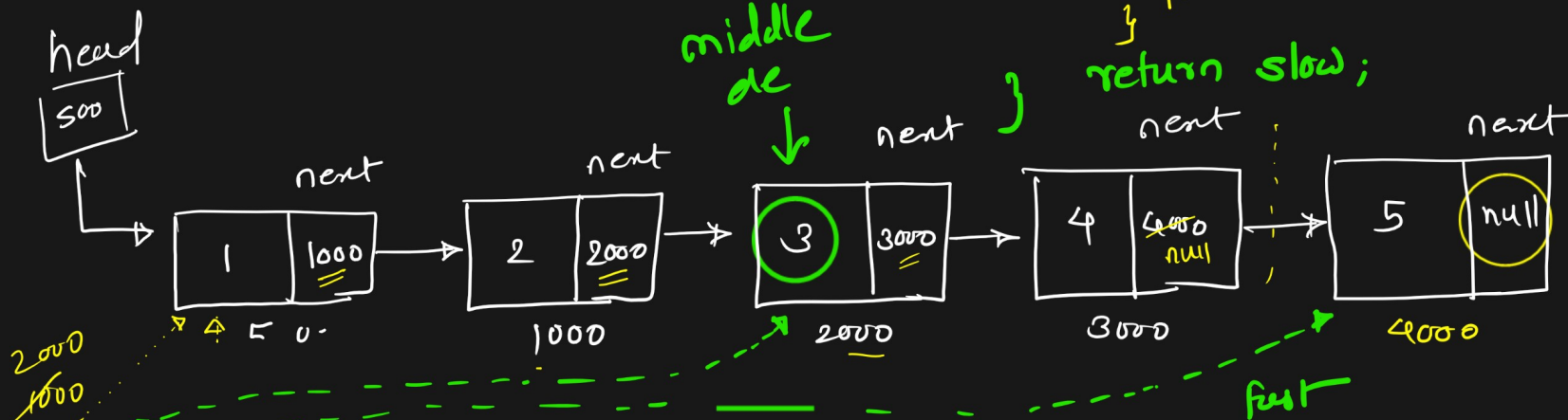
ListNode slow = head;

ListNode fast = head;

To Find out middle element of the linked list.
Use of Two Pointer Concept

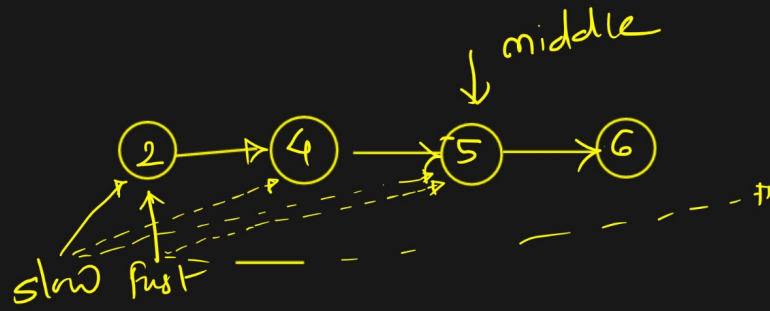
ListNode middleElement(ListNode head)

```
{
    ListNode slow = head;
    ListNode fast = head;
    while (fast.next != null & fast != null)
    {
        slow = slow.next;
        fast = fast.next.next;
    }
    return slow;
}
```



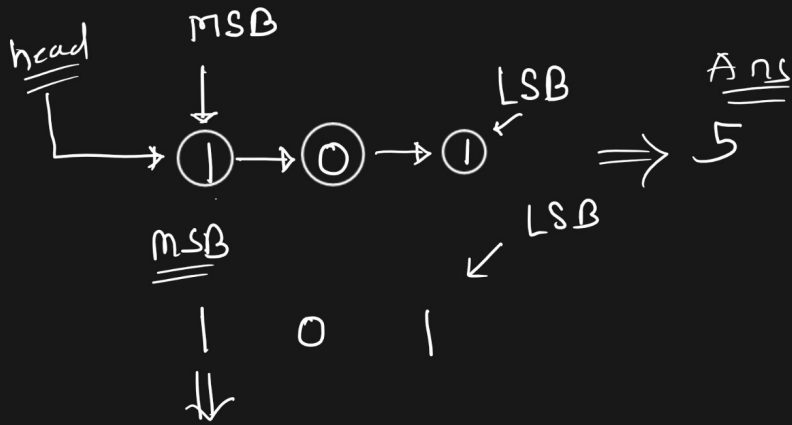
slow	fast
500	500
1000	2000 ✓
2000	4000 ✓

null



slow	fast
2	2
4	5
<u>middle 5</u>	null

② LeetCode Pbm No :- 1290 :- Convert binary linked list element to decimal

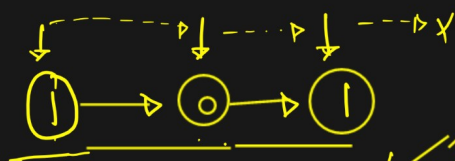


$$\begin{array}{c} \text{MSB} \swarrow \quad \nwarrow \text{LSB} \\ 1011 \\ \hline 8421 \end{array} \Rightarrow (1 \times 2^3) + (0 \times 2^2) + (1 \times 2^1) + (1 \times 2^0)$$

$$\Rightarrow 8 + 0 + 2 + 1$$

$$\Rightarrow \underline{\underline{11}}$$

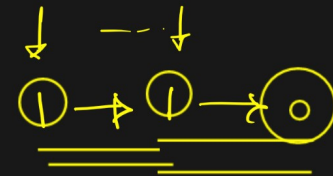
$$(1 \times 2^2) + (0 \times 2^1) + (1 \times 2^0) = 4 + 0 + 1 = 5$$



$$\text{ans} = 0$$

$$\text{ans} = 1 + (0 \times 2) \Rightarrow \underline{\underline{1}}$$

$$\text{ans} = \text{node value} + (\text{ans} \times 2)$$



$$\text{ans} = 0$$

$$\text{ans} = 1 + (0 \times 2) \Rightarrow \underline{\underline{1}}$$

$$\text{ans} = 1 + (1 \times 2) \Rightarrow \underline{\underline{3}}$$

$$\text{ans} = 0 + (3 \times 2) \Rightarrow \underline{\underline{6}}$$

$$\text{ans} = 0 + (1 * 2) \Rightarrow \underline{\underline{2}}$$

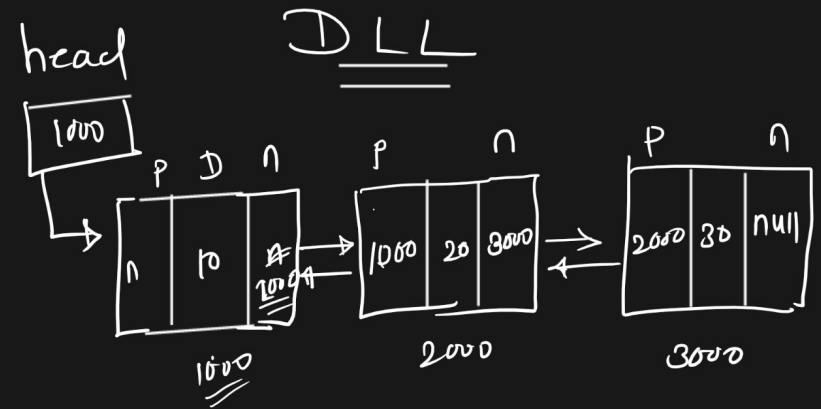
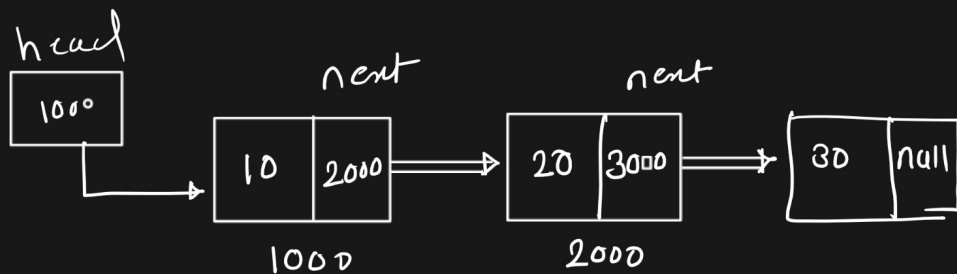
$$\text{ans} = 1 + (2 * 2) \Rightarrow \underline{\underline{5}}$$

```

int binarytodecimal(NodeList head)
{
    NodeList temp = head;
    int ans = 0;
    while(temp != null)
    {
        ans = temp.data + (ans * 2);
        temp = temp.next;
    }
    return ans;
}

```

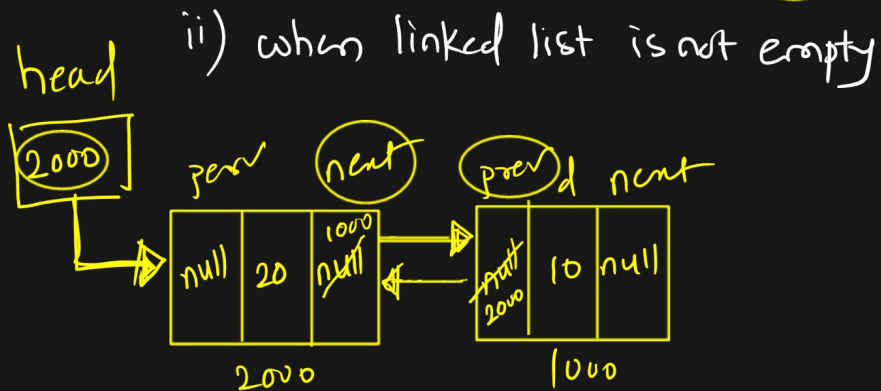
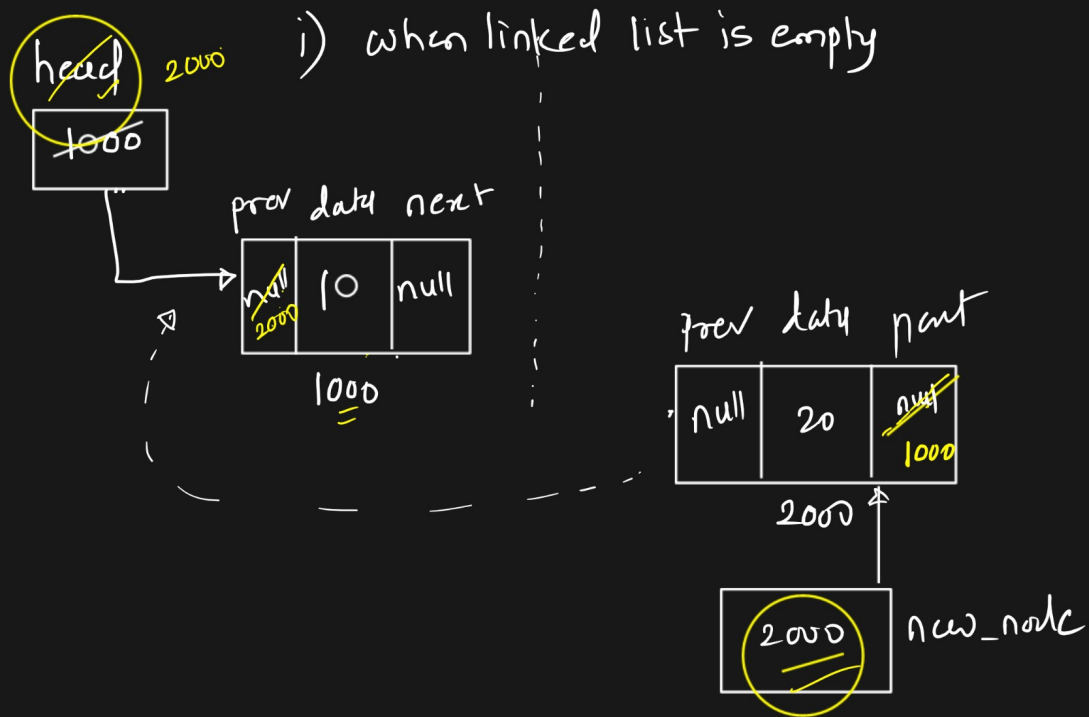
* Double Linked List:-



SLL $\Rightarrow$ Direction Forward type (single)

SLL

A) Insert At First



```
class Node
{
    int data;
    Node next;
    Node prev;

    Node (int num)
    {
        data = num;
        next = null;
        prev = null;
    }
}
```

```
class DLinkedList
{
    Node head;

    DLinkedList ()
    {
        head = null;
    }

    void insert_At_First (int num)
    {
        Node new_node = new Node (num);
        if (head == null)
        {

```

```

    } head = new_node;

```

```

else

```

```

{

```

```

    new_node->next = head;

```

```

    head, prev = new_node;

```

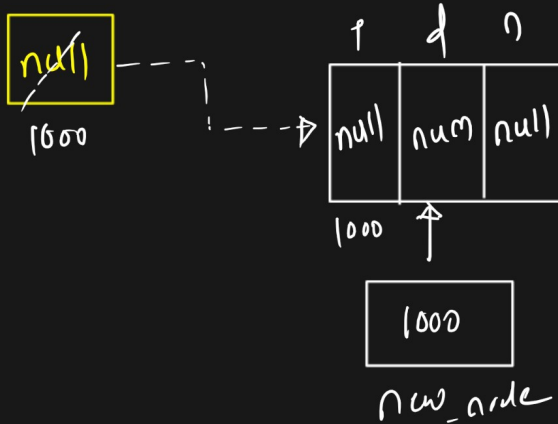
```

    } head = new_node;
}

```

B) Insert At Last :-

i) when linked list is empty
head



ii) when linked list is not empty

```

void insert_at_last ( int num)

```

```

{

```

```

    Node new_node = new Node(num);

```

```

    if ( head == null)

```

```

    {

```

```

        head = new_node;
    }

```

```

else

```

```

{

```

```

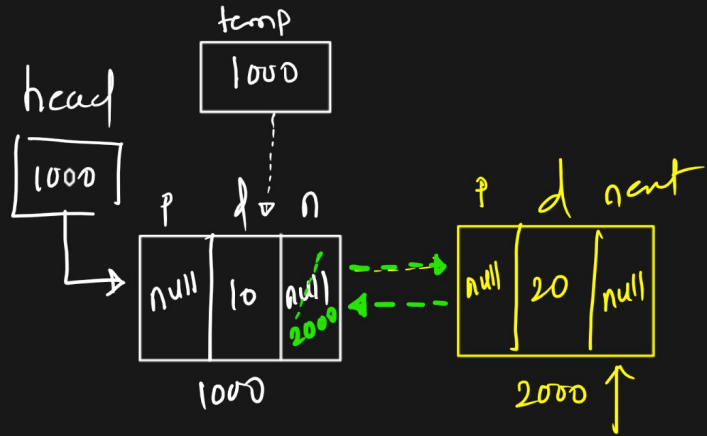
    Node temp = head;

```

```

    while (temp->next != null)

```



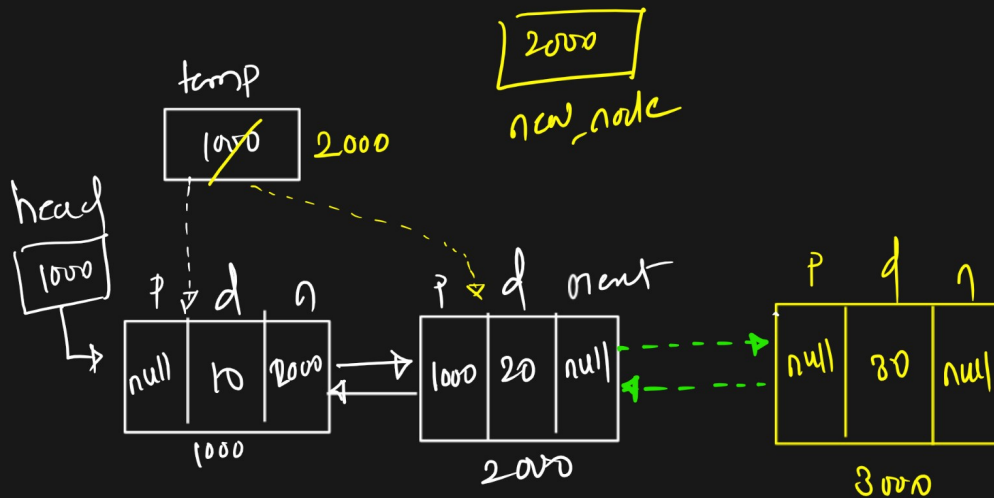
Data Structure Traversal

Array	Index
LL	pointer variable temp

```

{
    temp = temp->next;
}
temp->next = new_node;
new_node->prev = temp;
}

```



C) Delete_At_First:-

head
[null]

when linked list is empty



```

void delete_At_First ( )
{

```

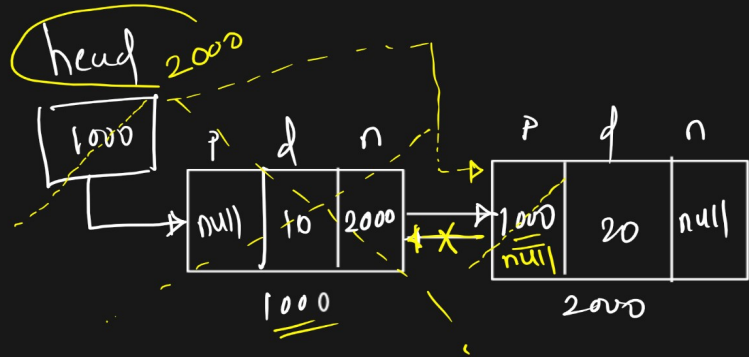
```

    if (head == null)
    {
        s.o.p (" Empty ");
        return;
    }

```

head = head->next;

ii) when linked list is not empty



who is point to first
node

head, 2000->prev

²⁰⁰⁰
head->prev = null;

D) Delete_At_Last

i) when linked list is empty

head

null

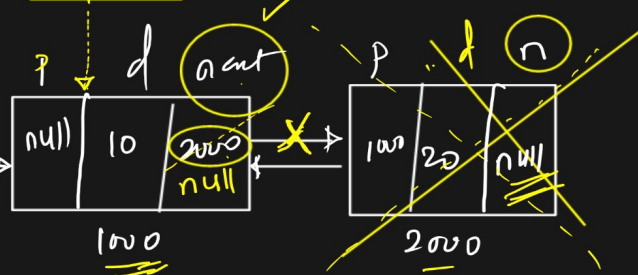
ii)

head

1000

temp

2000



temp 1000 2000

void delete_At_Last ()

```
{
    if (head == null)
    {
        s.o.p. ("Empty");
    }
    return;
}
```

Node temp = head;

```
while (temp->next->next != null)
{
    temp = temp->next;
}
```

²⁰⁰⁰
temp->next = null;

